

Number to Word Program(0-999)

PART 1

What the program actually does

The page asks the visitor for one whole number from 0-999, then builds an English sentence for it out of three pre-written word lists. There is no real "conversion math" — only range checks that decide *which list (or combination of lists)* to pull words from, plus some string-gluing.

The five possible outcomes for any input a:

- a is not a number
- a > 999 → Out of range
- 0-99 → one or two words
- 100 → "One Hundred"
- 101-999 → hundreds + remainder words

The three word banks it reads from

Every word the program ever prints comes from one of these three arrays. Pictured below: each box is array[index] next to the word that index holds.

arr1 — single digits, used for the ones place (index 0-9)

0	1	2	3	4	5	6	7	8	9
zero	one	two	three	four	five	six	seven	eight	nine

arr2 — the irregular teens (index 0-9 represents 10-19)

0 (10)	1 (11)	2 (12)	3 (13)	4 (14)	5 (15)	6 (16)	7 (17)	8 (18)	9 (19)
ten	eleven	twelve	thirteen	fourteen	fifteen	sixteen	seventeen	eighteen	nineteen

arr3 — the tens words (index 2-9 represents 20-90; index 0-1 are deliberately blank)

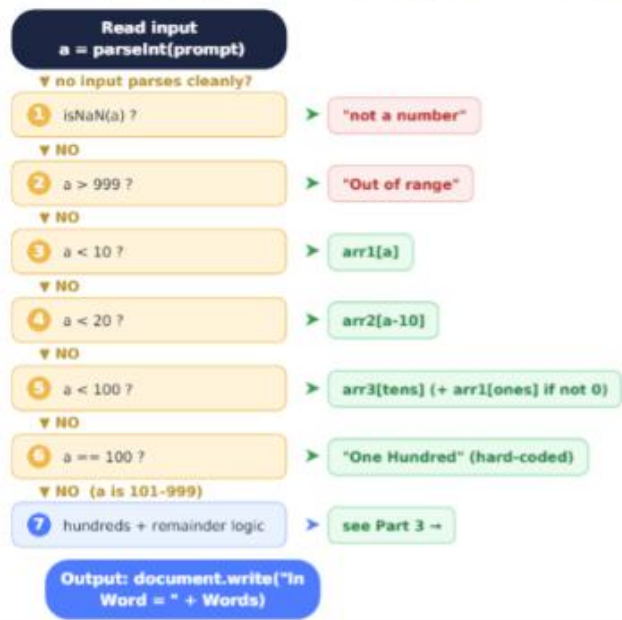
0	1	2 (20)	3 (30)	4 (40)	5 (50)	6 (60)	7 (70)	8 (80)	9 (90)
(unused)	(unused)	twenty	thirty	forty	fifty	sixty	seventy	eighty	ninety

Indices 0 and 1 of arr3 are left as empty strings on purpose — the code comment even explains why: *tens words only start at twenty, but array indexing starts at zero, so the first two slots are placeholders nobody ever reads.*

PART 2

The decision staircase

Read top to bottom. Every diamond is one if check from the code, in the exact order it appears. A **YES** answer peels off to the right into an outcome box and the program stops; a **NO** answer drops straight down to the next question.



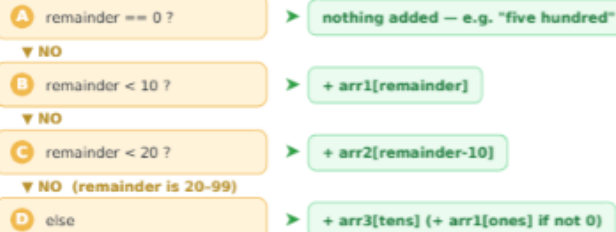
PART 3

Zooming in: numbers from 101 to 999

This is the branch with the most moving parts, so it earns its own diagram. The hundred-digit is always handled the same way; only the leftover two-digit remainder decides which of three patterns runs.

```
arr1[ [a/100] ] + "hundred" + remainder = a % 100
```

Then the remainder (0-99) is sentence-built by its own three-way split:



Worked examples, start to finish

Seven inputs, traced through the staircase from Parts 2-3.

a = 7	single digit
isNaN? no → >999? no → <10? yes → arr1[7] = "seven"	
In Word = seven	
a = 15	irregular teen
isNaN? no → >999? no → <10? no → <20? yes → arr2[15-10] = arr2[5] = "fifteen"	
In Word = fifteen	
a = 47	two-word tens
... → <100? yes → arr3[4] = "forty" + 47%10=7≠0 → arr1[7] = "seven"	
In Word = forty seven	
a = 40	exact tens, no ones word
<100? yes → arr3[4] = "forty" + 40%10=0 → skip arr1[]	
In Word = forty	
a = 100	special-cased exactly
=100? yes → hard-coded string "One Hundred"	
In Word = One Hundred (note the capitals — every other branch prints lower-case)	
a = 905	hundreds + lone single digit
a>100 → arr1[9]="nine"+"hundred" + remainder=5, <10 → arr1[5]="five"	
In Word = nine hundred five	
a = "abc" / a = 1500	the two failure paths
parseInt("abc") = NaN → isNaN? yes	
not a number	
a = 1500 → isNaN? no → >999? yes	
Out of range	